

Pràctica OpenMPI

FeedForward Neural Network

Computació d'Altes Prestacions. MatCAD

Natan Sisoev (1706198) i Ferran Villarta (1704051)

14 de gener de 2026

Sumari

1	Introducció	2
1.1	Motivació i Objectius	2
1.2	Conceptes Fonamentals	2
1.3	Eines i Tecnologies	3
1.4	Arquitectura del Projecte	4
2	Paral·lelització	5
2.1	Model de Xarxa Neuronal	5
2.2	Model de Programació MPI	6
2.3	Paral·lelisme de Dades	6
2.4	Agregació de Gradients	7
2.5	Implementació	8
3	Automatització	13
3.1	Scripts	13
3.2	Reproduïbilitat	14
3.3	Espai de Paràmetres	15
3.4	Mètriques	15
4	Resultats	17
4.1	Resultats numèrics	17
4.2	Resultats gràfics	19
4.3	Strong & Weak Scaling	21
4.4	Limitacions i anàlisi de colls d'ampolla (<i>bottleneck</i>)	22
4.5	Treball Futur	22
5	Annex	24
5.1	Taules Completes de Resultats	24
5.2	Codi	32
5.3	Scripts	35

1 Introducció

1.1 Motivació i Objectius

L'entrenament de xarxes neuronals és intensiu i, a mesura que els datasets es fan més grans, el temps d'entrenament també augmenta de forma proporcional. La solució a aquest problema és fer ús de la programació paral·lela i de diverses optimitzacions que es poden fer per reduir aquesta duració. Com ja hem fet en les altres pràctiques, en aquest informe expli em detalladament els canvis que hem fet al codi, la motivació darrera d'aquests canvis i els resultats.

Com a objectius generals de la pràctica, destaquem els següents.

- Paral·lelitzar l'entrenament de la xarxa neuronal.
- Mesurar i fer un anàlisi de l'speedup i eficiència.
- Analitzar l'escalabilitat de la paral·lelització i del codi.
- Extreure conclusions de tot el procés.

Com a objectius secundaris, intentarem fer tot de la manera més pragmàtica i sistemàtica possible, sense haver de fer desenes d'execucions a mà, anar canviant cada cop paràmetres del codi o altres tasques que es poden automatitzar. Intentarem extreure conclusions i trobar explicacions per tots els resultats

1.2 Conceptes Fonamentals

Igual que en les altres pràctiques, és fonamental tenir coneixement dels conceptes fonamentals següents que farem servir al llarg de les explicacions de la pràctica.

- **Accuracy:** $acc = n^o \text{ encerts} / n^o \text{ total}$, mesura com d'ajustades son les prediccions a les etiquetes reals (en tant per u)
- **Speedup:** $S = T_{seq} / T_{par}$, mesura com més ràpid és la versió paral·lelitzada respecte la seqüencial

- **Eficiència:** $E = S/p$, mesura com de bona és la paral·lelització comparant-la amb el màxim ideal teòric
- **Llei d'Amdahl:** $S_{\max} = 1/((1 - f) + f/p)$, reflecteix l'existència d'una part del codi inherentment seqüencial
- **Llei de Gustafson:** $S(p) = p - \alpha(p - 1)$, reflecteix com la part seqüencial es fa més petita a mesura que augmenta la càrrega de treball paral·lelitzable
- **Període d'entrenament:** fase del programa on, per cada *epoch*, es dur a terme la propagació endavant i endarrera de totes les imatges optimitzant cada cop els pesos i biaixos de cada neurona
- **Període de testeig:** fase del programa on, un cop entrenada la xarxa neuronal, es fa la predicció sobre un dataset disjunt per tal de calcular la precisió i el nivell d'aprenentatge

També és important tenir uns coneixements bàsics del funcionament de les xarxes neuronals.

1.3 Eines i Tecnologies

Per dur a terme aquesta pràctica hem fet ús de diverses eines que explicarem a continuació.

- **OpenMPI:** Implementació lliure del model de pas de missatges (MPI, Message Passing Interface), utilitzada per a la programació paral·lela en entorns distribuïts. Permet que diferents processos d'un programa intercanviïn informació (que s'ha de passar explícitament) i treballin conjuntament sobre tasques grans de manera eficient.
- **SLURM:** Sistema de gestió de recursos en clusters, per assignar nodes, gestionar cues i distribuir càrregues eficientment.
- **Python:** Llenguatge de programació que s'ha fet servir per automatitzar l'execució del programa amb diferents paràmetres (especialment, el fitxer `run.py`).

- **Make:** Petit script que conté la crida exhaustiva de totes les combinacions dels paràmetres, fent ús del programa de Python.

Cal remarcar que el codi principal està fet en el llenguatge C.

1.4 Arquitectura del Projecte

L'estructura del codi i de tot el programa és modular, per tal de facilitar la separació de tasques independents en els diferents fitxers.

- **main.c/h:** programa principal
- **training/:** algorisme de backpropagation
- **layer/**, **initialize/**, **randomizer/**, **common/**, **configuration/:** funcions i fitxers auxiliars
- **datasets/:** fonjunts de dades (`optdigits.tra`, `optdigits.cv`)
- **mpi.sub:** *scheduler* principal que envia la tasca al servidor, se li poden passar paràmetres (número d'*epochs*, de neurones i quantes repeticions) que guardarà, junt amb altres informacions, a la metadata del fitxer de sortida
- **run.py:** script de Python que fa la crida al *scheduler* al qual se li poden passar diversos paràmetres com arguments (com ara `python3 run.py -num_epochs 100 -partition new-nodo.q`)
- **makefile:** petit script per executar tot el programa i obrir el fitxer d'*output* un cop acabat d'executar

Tot està programat de tal forma que no s'hagi de tocar codi per poder executar certes configuracions. Per exemple, per provar 200 èpoques amb 500 neurones en la capa oculta només cal cridar l'*scheduler* amb el paràmetre corresponent: `sbatch mpi.sub 200 500`. Altres canvis, com el fitxer de partició o el número de processos es poden canviar fent servir el fitxer de Python, com ara `run.py -partition new-nodo.q -nprocs 44`. Tot es pot fer des del directori principal del projecte, i els resultats es guardaran en la carpeta `OUT/`.

2 Paral·lelització

2.1 Model de Xarxa Neuronal

Com s'ha comentat en la introducció de l'informe, l'objectiu és paral·lelitzar una xarxa neuronal. Per això, necessitem entendre una mica com funciona i quins aspectes d'aquesta podem explotar per reduir el temps total

En el nostre cas, tenim una *Multi-layer Perceptron* (MLP), que està constituïda per diferents capes amb neurones. La primera capa ve dada per les dades d'entrada, i la última per les prediccions. Cada neurona té una activació i una funció de pes que propaga a totes les neurones de la capa següent. D'aquesta manera, a la dada inicial se li apliquen molts càlculs iterats per arribar a un sol número final, la probabilitat de ser aquella xifra. Aquesta fase s'anomena *forward propagation*.

Un cop fet la primera fase, es comparen els resultats amb el *ground truth*, és a dir les etiquetes reals. Amb el mètode de descens de gradient, es re-calculen els pesos i biaixos de cada neurona per tal d'ajustar la predicció al valor real. Aquesta fase s'anomena *backward propagation*.

Aquestes dues etapes constitueixen un pas de l'entrenament per a una dada inicial. Donades n imatges per entrenar, l'algoritme haurà de repetir aquest procés n cops, cada vegada millorant els pesos i ajustant-los per tal de produir prediccions més bones.

Donades les dades inicials, es veu que hi haurà molta diferència entre l'estat de la xarxa a la primera iteració que a la última: la darrera serà molt més bona. Per això, el que es fa normalment és repetir aquest procés un altre cop, encapçalant una època (o *epoch* en anglès). D'aquesta manera, la xarxa torna a veure les mateixes dades tants cops com èpoques. El que s'espera és que la xarxa vagi millorant amb cada iteració. Un aspecte important de les èpoques és que cada una es construeix a sobre de l'anterior, donant-nos ja pistes sobre si podem paral·lelitzar-les o no.

Un cop explicat el funcionament general de la xarxa neuronal, fem una sèries d'observacions. Cada imatge individual afecta "poc" al resultat final. El seu funcionament es basa en bases de dades d'entrenament massives, de forma que els petits aprenentatges de cada

imatge, agregats sobre totes les dades, habilitin una certa capacitat de generalització. No passa el mateix amb les èpoques (com ja hem comentat): cadascuna evoluciona a partir de l'última, no es poden canviar d'ordre. També cal remarcar que el pes real de la computació es troba en el caire iteratiu de l'algoritme: una iteració individual amb tots els càlculs que comportin, amb les tecnologies actuals, és molt ràpida, on es suma més temps és repetir-los per totes les imatges.

2.2 Model de Programació MPI

Ara passarem a explicar per sobre la filosofia de la programació MPI, és dir *Message Passing Interface*. És un model de programació amb memòria distribuïda, és dir que els diferents processos es distribuïran la memòria. Hi ha altres maneres d'obtenir dades en comú (compartides) entre tots els processos, però la més característica és el pas d'informació.

L'estructura habitual és repartir la mateixa tasca que s'ha de fer sobre moltes dades entre diferents processos. Hi ha un procés principal, que és el *master*, i tots els altres, els *slaves*. En funció de quin procés s'està executant, fins i tot es poden repartir el treball, és dir que cada procés faci tasques diferents.

El pas de dades es fa a través de comandes específiques enfocades en enviar o rebre variables. La versió col·lectiva d'aquests traspassos fa les dues alhora: enviar informació a totes els altres processos i també en rep dels altres (si és el cas).

Un cas d'ús típic és el producte escalar. Es reparteixen les posicions entre tots els processos, cada procés rep dos números que multiplica i torna al *master*. Aquest últim s'encarrega de fer la suma fent un altre cop reducció o fent servir altres mètodes. Amb aquest exemple crec que ja és natural el tipus de paral·lelització que cal aplicar en el nostre cas.

2.3 Paral·lelisme de Dades

Com ja s'ha motivat detalladament en les sub-seccions anteriors, el model ideal de paral·lelització en el cas de la nostra xarxa neuronal és el paral·lelisme de dades. Repartirem

totes les dades (imatges) entre els processos, aquests faran la seva part de la feina i retornaran els resultats i els posaran en comú.

Primer, veiem per què funciona aquesta separació de dades. Com hem explicat en les observacions de l'explicació del model de xarxa neuronal, sabem que cada imatge individual no aporta un canvi significatiu al model. Per això, podríem pensar que repartir les imatges entre tots els processos i després, d'alguna forma, agregar els resultats resultaria en una xarxa neuronal semblant a la original. Fins i tot, podríem dir que en certs casos és millor aquesta estratègia.

Llavors, sabem que podem separar les dades, la pregunta és com ho fem. A priori, cap procés té preferència sobre els altres, tots són iguals i han de fer el mateix treball: entrenar la seva xarxa neuronal pròpia. Per tant, la manera en què repartirem les dades serà equitativa entre els processos. Si la divisió no és entera, repartirem els sobrants entre els primers processos, com a màxim la diferència de treball serà d'una imatge. A continuació un exemple de partició.

Nota: s'ha intentat fer la càrrega de patrons per només el procés *master*, i que aquest faci un *scatter* als altres processos per tal d'evitar que tots facin una càrrega del *dataset* complet quan realment només necessiten una petita part. Tot i això, no s'han trobat diferències en el temps a diversos processos per una possible cancel·lació de temps (entre la càrrega i la distribució), per tant no s'han inclòs els resultats en l'informe.

- Si tenim 60000 patrons i 8 processos: cada procés rep 7500 patrons.
- Si hi ha residu (ex: 60003 patrons), els primers processos reben 1 patró extra.
- Això garanteix balanceig de càrrega òptim.

Finalment, tenim les dades repartides i cada procés amb la seva xarxa neuronal entrenada. Com podem seguir endavant amb la següent època?

2.4 Agregació de Gradients

Un cop tenim les n *mini*-xarxes neuronals (*mini* perquè s'han entrenat sobre un subconjunt de totes les dades), tocaria comunicar els resultats als altres *slaves*, o només al *master*.

Creiem que no cal molta explicació i que no és molt suposar que la millor solució és fer la mitjana de totes les xarxes neuronals (ens referim als pesos i biaixos). Una alternativa seria quedar-nos amb aquella que aconseguix millors resultats (s'haurien de testejar sobre tot el conjunt de dades per separat i comparar-les), però pensem que aquesta solució se surt del domini d'aquesta pràctica enfocada en paral·lelització i no tant optimització de la xarxa.

Per tant, un cop cada procés té la seva xarxa entrenada, cal fer la mitjana de totes i tornar a repartir-les entre els processos per poder seguir amb la següent època. Afortunadament, no cal fer el pas de dades manual de cada procés a tots els altres, ja que MPI implementa un concepte útil de comunicació col·lectiva (mencionat també anteriorment). Així, fent servir la funció `MPI_Allreduce` amb la opció `MPI_SUM`, seguit d'una simple divisió, aconseguim propagar els resultats locals a tots els processos. Un cop fet això, es pot seguir amb la següent època, al final de la qual es tornaran a compartir els resultats.

2.5 Implementació

En aquest apartat anirem explicant cada canvi que hem fet al codi, motivat per les explicacions anteriors i validat en els apartats següents de resultats. No incloem aquí el codi sencer, només les parts que considerem essencials per entendre el funcionament general (aquest es troba en l'annex).

Com qualsevol programa MPI, primer cal inicialitzar la secció paral·lela. Tot seguit, guardem el nombre total de procés i l'identificador del procés actual a través d'unes variables definides a `main.h`.

```
1 int main(int argc, char** argv) {
2     MPI_Init(&argc, &argv);
3
4     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5     MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
6
7     /* inicialització + entrenament + testeig */
8
9     MPI_Finalize();
10
11     return 0;
12 }
```

Per tal de poder passar dinàmicament alguns paràmetres fonamentals al programa sense

haver de crear arxius de configuració cada cop o canviar manualment el valor, hem fet que el programa accepti 2 paràmetres: el nombre de èpoques i neurones. Tampoc hem volgut perdre la funcionalitat de passar un fixer de configuració personalitzat, així que si hi ha un sol argument (a part del nom del fitxer que ve per defecte), l'agafarà com el nom del fitxer de configuració. Si en venen dos arguments, els entendre com els dos paràmetres que hem posat. Per això, si es vol indicar un, també s'ha d'indicar l'altre per evitar confusió amb el nom del fitxer de configuració.

```
1 int main(int argc, char** argv) {
2     /* ... */
3
4     if (argc == 2) {
5         readConfiguration(argv[1]);
6     } else {
7         readConfiguration("configuration/configfile.txt");
8     }
9
10    if (argc == 3) {
11        num_epochs = atoi(argv[1]);
12        num_neurons[1] = atoi(argv[2]);
13    }
14
15    /* ... */
16 }
```

Com després voldrem fer proves sobre la millora que implica la paral·lelització, expliquem en aquest apartat ràpidament el funcionament de la mesura del temps. Bàsicament, hi ha una funció de MPI, `MPI_Wtime` que retorna el rellotge de la unitat de processament de la GPU actual. Mesurant el temps abans i després del còmput i fent la resta aconseguim la durada total d'aquella secció del programa. Donats tots els temps de cada procés, els agregarem considerant el màxim d'aquests, ja que el programa és igual de lent que el més lent dels processos. Finalment, guardem els temps i els imprimim per pantalla al final del programa (només ho fa el procés *master*).

```
1 void train_neural_net() {
2     double t0 = MPI_Wtime();
3
4     /* ... */
5
6     double local = MPI_Wtime() - t0;
7     MPI_Reduce(&local, &train_t, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
8 }
```

```
9
10 void test_nn() {
11     double t0 = MPI_Wtime();
12
13     /* ... */
14
15     double local = MPI_Wtime() - t0;
16     MPI_Reduce(&local, &test_t, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
17 }
18
19 int main(int argc, char** argv) {
20     /* ... */
21
22     if (rank == 0) {
23         printf("RIGHT %d\n", total);
24         printf("ACCUR %6.4f\n", (float) total / num_test_patterns);
25         printf("TRAIN %6.4f\n", train_t);
26         printf("TEST %6.4f\n", test_t);
27         printf("TOTAL %6.4f\n", train_t + test_t);
28     }
29
30     /* ... */
31 }
```

Continuem amb la part d'entrenament. Al començament de tot, volem calcular el repartiment òptim dels *patterns*, que ho fem seguint la fórmula explicada anteriorment. Un cop cada thread calcula el tros de dades que li toca, el *master* decideix la ordenació aleatòria inicial per cada època. Aquesta tasca la fa només un procés per tal d'evitar repetició de patrons entre processos (cas en que cada procés té la seva ordenació i potser es solapen ja que els índex ja s'han repartit). Per tant, decideix la ordenació aleatòria i la comunica als *slaves*. Cadascun d'aquests itera sobre les seves imatges i fa el procés complet (les dues fases explicades a l'inici de l'informe). Al final de la època, s'itera sobre totes les capes i s'actualitzen seguint la lògica explicada.

```
1 void train_neural_net() {
2     /* ... */
3
4     int extra = num_training_patterns % num_procs;
5     int from = rank * (num_training_patterns / num_procs) + ((rank < extra) ? rank :
6         extra);
7     int to = from + (num_training_patterns / num_procs) + ((rank < extra) ? 1 : 0);
8
9     /* ... */
```

```
10     for (int it = 0; it < num_epochs; it++) {
11         if (rank == 0) {
12             for (int p = 0; p < num_training_patterns; p++)
13                 ranpat[p] = p;
14
15             for (int p = 0; p < num_training_patterns; p++) {
16                 int x = rando();
17                 int np = (x * x) % num_training_patterns;
18                 int tmp = ranpat[p];
19                 ranpat[p] = ranpat[np];
20                 ranpat[np] = tmp;
21             }
22         }
23
24         MPI_Bcast(ranpat, num_training_patterns, MPI_INT, 0, MPI_COMM_WORLD);
25
26         for (int i = from; i < to; i++) {
27             int p = ranpat[i];
28             feed_input(p);
29             forward_prop();
30             back_prop(p);
31             update_weights();
32         }
33
34         for (int l = 0; l < num_layers - 1; l++) {
35             long size = num_neurons[l] * num_neurons[l+1];
36
37             MPI_Allreduce(MPI_IN_PLACE, lay[l].out_weights, size, MPI_FLOAT, MPI_SUM,
38                           MPI_COMM_WORLD);
39             for (long i = 0; i < size; i++)
40                 lay[l].out_weights[i] /= num_procs;
41
42             MPI_Allreduce(MPI_IN_PLACE, lay[l].bias, num_neurons[l], MPI_FLOAT, MPI_SUM,
43                           MPI_COMM_WORLD);
44             for (int i = 0; i < num_neurons[l]; i++)
45                 lay[l].bias[i] /= num_procs;
46         }
47     }
48 }
```

Per la part de testeig és molt semblant, fins i tot més fàcil. Es reparteixen tots els patrons de forma seqüencial, cada procés fa les proves acumulant els encerta i, al final, es sumen tots aquests.

```
1 void test_nn() {
2     /* ... */
```

```
3
4     int extra = num_test_patterns % num_procs;
5     int from  = rank * (num_test_patterns / num_procs) + ((rank < extra) ? rank : extra);
6     int to    = from + (num_test_patterns / num_procs) + ((rank < extra) ? 1 : 0);
7
8     for (int i = from; i < to; i++) {
9         for (int j = 0; j < num_neurons[0]; j++)
10             lay[0].actv[j] = rSet[i][j];
11
12         forward_prop();
13         printRecognized(i, lay[num_layers - 1]);
14     }
15
16     int global_total = 0;
17     MPI_Allreduce(&total, &global_total, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
18     total = global_total;
19
20     /* ... */
21 }
```

3 Automatització

3.1 Scripts

Per tal d'automatitzar el procés d'avaluació del programa, s'han creat uns scripts que serveixen per diferents tasques. No s'explicarà cada línia d'aquests ja que no és la part principal de la pràctica, però sí s'explicarà el seu funcionament i com interactuar amb ells. Aquests s'inclouen en la seva totalitat al final de l'informe.

`mpi.sum`

Primer, i més important, tenim l'*scheduler* `mpi.sub`. Aquest rep 3 paràmetres: nombre d'èpoques, de neurones i de repeticions. Abans de compilar, guarda tots aquests paràmetres (i més) en forma de metadata en el fitxer de sortida. També s'inclouen altres dades essencials per la identificació precisa de l'execució.

Finalment, compila i executa el programa passant per paràmetres els primers dos que acabem d'explicar. L'execució la fa tantes vegades com calgui, separant cadascun per pel caràcter »"repetit molts cops i seguit pel comptador de les execucions. Això servirà per tenir resultats més o menys estables i intentar eliminar al màxim possible la variabilitat del servidor.

`run.py`

Aquest és el fitxer que més es farà servir. És una mena de *wrapper* del *scheduler* habilitant també el canvi automàtic del fitxer de partició i número de processos (els quals no es poden passar per paràmetre ni per variables d'entorn al `mpi.sub`). Més convenient que el *script* anterior, es poden passar els arguments per nom, és dir indicant primer el seu nom i després el seu valor, per exemple `-num_epochs 100` indicant que l'execució s'ha de fer amb cent èpoques. També es pot passar un nom personalitzat per tal de diferenciar uns fitxers de sortida específics (els d'aquella execució) de tots els altres. Com a funcionalitat addicional i per defecte, un cop finalitzat el *job*, s'obre el fitxer de sortida en Visual Studio Code. Per desactivar aquesta comportament cal activar el flag `-no-open`.

Amb totes aquestes dades, el programa canvia el fitxer `mpi.sub` per reflectir-les i envia la tasca d'execució, també amb els paràmetres apropiats. Si no s'indica el contrari, el programa seguirà esperant fins que la tasca enviada s'acabi, moment en què obrirà el fitxer de sortida.

`makefile`

Tercer script personalitzat, el `makefile`. Aquest el farem servir per execucions ràpides amb paràmetres per defecte (o *hard-coded*) executant la comanda `make`.

També hem programat una altra opció, la `test1`. Aquesta crea una malla de cerca a través de múltiples *fors* anidats, per tal d'explorar totes les combinacions possibles dels paràmetres de les llistes donades en la capçalera del fitxer. Envia les tasques a través del script de Python amb la opció `-no-open` per tal d'evitar que s'obrin tots els fitxers.

Posteriorment, els fitxers de sortida es poden consultar en la carpeta `OUT/` i analitzar.

`results.ipynb`

Quart i últim fitxer creat, que no és realment un *script*, per tant només l'incloem en aquest apartat per no crear-ne de nous. Aquest notebook de Jupyter es pot fer servir per analitzar els resultats del `test1`. Concretament, itera sobre tots els fitxers i guarda els resultats. En cas que l'execució no disposi de cap execució vàlida, ho imprimeix per pantalla. A la primera execució, n'han sortit dos fitxers, els quals hem executat manualment i hem reemplaçat els resultats.

Amb el diccionari total de resultats recollit, en les cel·les que segueixen hi ha el codi per fer diferents anàlisi del rendiment, incloent el codi per generar les gràfiques de la secció següent i els resultats.

3.2 Reproduïbilitat

Per tal de reproduir els experiments i resultats explicats en aquesta pràctica, és tan fàcil com executar la comanda `make test1`, la qual enviarà totes les execucions a la cua. Posteriorment, es poden consultar tots els resultats en la carpeta `OUT`, on cada fitxer conté el paràmetres d'execució en la seva capçalera.

3.3 Espai de Paràmetres

Per l'espai de paràmetres de cerca, s'ha seguit principalment el document de referència proporcionat pel professorat de l'assignatura. Per això, l'elecció de les quantitats tan específiques (però sense un patró clar) de processos queda una mica ofuscada, però per evitar confusions hem seguit la mateixa successió creixent. Entre les particions tenim els dos fitxers que també s'indiquen en la referència, igual que el nombre d'èpoques. El nombre de neurones està fixe a 250, però nosaltres també hem provat per 135. La malla final és, doncs, la següent (que es troba en el fitxer `makefile`).

```
1 PARTITIONS := nodo.q new-nodo.q
2 NPROCS := 2 4 6 8 10 12 16 20 24 32 36 40 48 64
3 NUM_EPOCHS := 10 100
4 NUM_NEURONS := 135 250
```

Nota molt important: és vital ser conscient que la variabilitat del servidor és molt present en totes les execucions i pot influir al rendiment del programa i la seva avaluació. Per això, com a pas preventiu hem repetit cada combinació 5 cops. Alguns d'aquests donen temps molt més alts que la mitjana (o esperat), amb alguns fins i tot acabant sense poder assignar el nombre de processos que s'ha demanat. Repetint qualsevol combinació del *test1* a part, es veurà clarament que el temps que s'aconsegueix real és molt més petit (en alguns casos amb una diferència de més del 25% del temps tota). Creiem que això es deu al fet que tots els nodes del servidor estan ocupats amb les altres tasques del test, fent que cada execució trigui més.

3.4 Mètriques

Definir temps total, train/time, breakdown, accuracy, speedup, eficiència.

Pel que fa les mètriques de les execucions, en centrarem en 4 principals: el temps total, el nombre d'encerts, l'*speedup* i l'eficiència. El primer i segon és trivial, tot i que cal notar que el nombre d'encerts variarà una mica en funció dels nodes i, fins i tot de la forma de programar la paral·lelització (tot i ser conceptualment equivalent). Sempre tindrem en compte els encerts per assegurar-nos que el codi funciona com volem. Al final, farem també unes reflexions sobre aquest aspecte de la pràctica. El temps ja s'ha comentat en la nota

anterior que no és molt fiable, però seguirem el mínim de les 5 execucions per evitar els *outliers* (que en molts casos son 2, fins i tot 3 execucions de les 5), és dir instàncies en què el temps total d'execució és anormalment alt, sense possibilitat de deure's a l'arquitectura del programa.

L'*speedup* i eficiència ja s'han explicat, però bàsicament mesuren com de millor va el programa (en quant a temps) respecte la versió en sèrie i com de bona és la programació respecte el líndar teòric, respectivament. Aquestes es calcularan en comparació al temps seqüencial que equival a la paral·lelització amb només 1 procés.

4 Resultats

Els resultats principals els presentarem amb una taula resum (totes les taules es poden consultar en l'annex) i uns gràfics agregats amb tots els resultats.

4.1 Resultats numèrics

La següent taula resumeix els resultats ensenyant només la quantitat òptima de processos en cadascun dels 8 casos generats per les combinacions de particions, èpoques i neurones.

Partició	Neurones	Èpoques	Nº Proc.	Temps	Encerts	Speedup	Eficiència
new-nodo.q	135	10	64	0.2338	641	13.2643	0.2073
new-nodo.q	135	100	64	2.1767	885	14.4413	0.2256
new-nodo.q	250	10	24	0.8025	849	7.3512	0.3063
new-nodo.q	250	100	64	7.8317	895	7.4349	0.1162
nodo.q	135	10	40	0.6546	775	15.8219	0.3955
nodo.q	135	100	40	7.3384	892	14.7079	0.3677
nodo.q	250	10	40	1.9913	827	9.6091	0.2402
nodo.q	250	100	48	22.4182	907	8.4203	0.1754

Taula 1: Millor configuració per temps en cada cas

Veiem que, tot i obtenir els millors temps, els *speedups* i eficiències no son gaire elevats. Mirem les taules amb el *top* de cadascuna d'aquestes mètriques.

Partició	Neurones	Èpoques	Nº Proc.	Temps	Encerts	Speedup	Eficiència
new-nodo.q	135	10	64	0.2338	641	13.2643	0.2073
new-nodo.q	135	100	64	2.1767	885	14.4413	0.2256
new-nodo.q	250	10	24	0.8025	849	7.3512	0.3063
new-nodo.q	250	100	64	7.8317	895	7.4349	0.1162
nodo.q	135	10	40	0.6546	775	15.8219	0.3955
nodo.q	135	100	40	7.3384	892	14.7079	0.3677
nodo.q	250	10	40	1.9913	827	9.6091	0.2402
nodo.q	250	100	48	22.4182	907	8.4203	0.1754

Taula 2: Millor speedup en cada configuració

Pel que fa l'*speedup*, s'aconsegueixen uns números satisfactoris, superant en almenys 7 vegades el temps seqüencial en tots els casos, però veiem que aquest s'aconsegueix pel màxim de processos quasi en tots els casos. Analitzem la taula de les eficiències.

Partició	Neurones	Èpoques	Nº Proc.	Temps	Encerts	Speedup	Eficiència
new-nodo.q	135	10	2	3.2115	876	0.9657	0.4828
new-nodo.q	135	100	2	33.5528	917	0.9369	0.4684
new-nodo.q	250	10	2	6.1559	895	0.9583	0.4792
new-nodo.q	250	100	4	32.9367	916	1.7679	0.4420
nodo.q	135	10	2	5.5925	876	1.8519	0.9260
nodo.q	135	100	2	51.5186	917	2.0950	1.0475
nodo.q	250	10	2	9.8299	895	1.9466	0.9733
nodo.q	250	100	2	97.3295	917	1.9395	0.9697

Taula 3: Millor eficiència en cada configuració

Veiem que per la partició *new-nodo.q* no s'aconsegueix pujar del 0.5 d'eficiència. En canvi, per l'altre fitxer si que s'arriba fins a més de 1! Això podria ser causa d'una execució seqüencial especialment lenta en aquell cas, fent que les versions paral·leles aconseguixin superar aquest límit teòric. El mateix podria explicar la diferència entre els dos fitxers: en el primer fins i tot la versió seqüencial va bastant ràpid, mentre que en el segon aquest triga considerablement més (més del doble).

En conjunt, les configuracions òptimes en temps apareixen en general amb **nprocs** elevats, però amb eficiències baixes, fet coherent amb un escalat típic quan la part paral·lelitzable del codi no és dominant o el cost de sincronització no és negligible (la darrera en el nostre cas). Els millors *speedups* també es donen a valors del màxim de processos disponibles, reforçant aquesta mateixa lectura. Les eficiències més altes, en canvi, apareixen a processos petits, com és habitual. Les eficiències superiors a 1 observades a **nodo.q** no impliquen superlinealitat real sinó un efecte de referència degut al temps seqüencial anòmalament elevat en aquest cas. La diferència entre **new-nodo.q** i **nodo.q** apunta a diferència de plataforma o condicions de càrrega, fet que afectaria els temps seqüencials i, per extensió, les mètriques relatives.

4.2 Resultats gràfics

Passem als gràfics. Separant per partició, disposem de quatre gràfics del temps (línia contínua) i dels encerts (línia discontinua) en funció del nombre de processos, un per a cada combinació de neurones i èpoques.

Partition: nodo.q

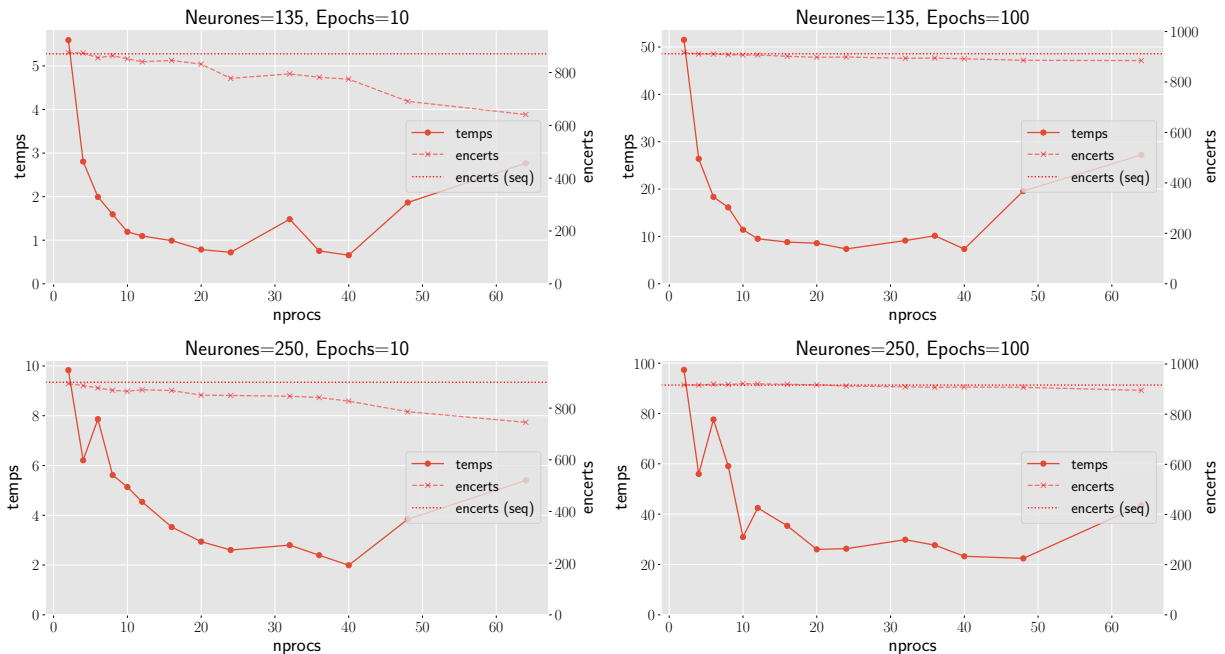


Figura 1: Partició nodo.q

En el cas de `nodo.q`, el temps disminueix de manera monotònica fins aproximadament 40 processos. A partir d'aquest punt el cost s'inverteix i augmenta. Aquest comportament és coherent amb un punt de saturació on el cost de comunicació i sincronització passa a dominar el càlcul útil. El límit pràctic observat al voltant de 48 processos pot correspondre a un límit de node o bé a un canvi de política d'assignació de recursos. No el considerem representatiu del rendiment en règim favorable.

Pel que fa als encerts, la diferència entre 10 i 100 èpoques és clara: amb 10 èpoques hi ha una lleugera degradació amb `nprocs`, mentre que amb 100 èpoques el resultat es manté pràcticament igual que el seqüencial. Això és positiu perquè indica que, un cop el procés d'entrenament ha convergit, la paral·lelització no introdueix pèrdues d'exactitud rellevants i per tant el temps es pot reduir sense comprometre el resultat final.

Partition: new-nodo.q

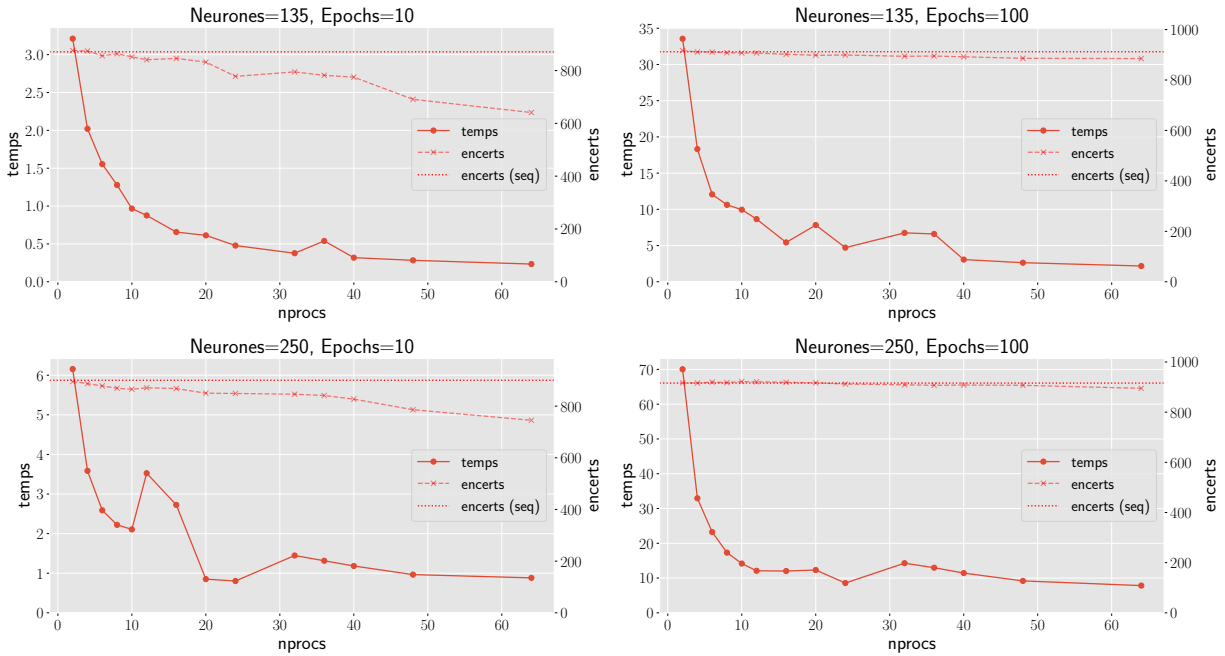


Figura 2: Partició new-nodo.q

En el cas de `new-nodo.q` el comportament és qualitativament el mateix, però el règim de millora s'estén fins a 64 processos. Això suggereix una plataforma amb recursos més favorable. De nou, el temps decreix amb `nprocs` fins al límit operatiu i els encerts romanen estables en el rang d'interès.

4.3 Strong & Weak Scaling

Una lectura raonable dels resultats permet parlar de *strong scaling* de forma bastant neta. Partint del cas seqüencial com a base, el temps d'execució decreix de manera aproximadament exponencial quan s'incrementa el nombre de processos fins a un cert llindar, moment en què el cost de comunicació i la saturació del node comencen a dominar. En la primera partició (`nodo.q`) aquest límit apareix cap als 40–48 processos. En l'altra partició actual (`new-nodo.q`) la mateixa corba continua fins a 64 processos, cosa que indica una arquitectura

o configuració més amable amb el paral·lelisme. En tots dos casos la mètrica d'encerts es manté estable per 100 èpoques i només es degrada lleugerament per 10 èpoques, la qual cosa fa que la decisió òptima de punts operatius no quedi sacrificada per la qualitat del model.

Pel que fa a *weak scaling*, es pot defensar que el comportament és admissible encara que no sigui perfecte. L'escenari en què s'augmenta el problema (en aquest cas, les èpoques o les neurones) proporcionalment al nombre de processos no arriba a saturar els encerts ni a degradar-los. Aquesta mena de *plateau* amb pendent suau abans de l'augment final es pot llegir com un espai on el cost de comunicació creix però no domina completament, i per tant la càrrega afegida encara s'amortitza.

4.4 Limitacions i anàlisi de colls d'ampolla (*bottleneck*)

Tot i que el model i el codi funcionen correctament per a l'escala estudiada, diversos factors limiten el rendiment i expliquen els colls d'ampolla observats. La relació entre comunicació i càlcul és clau: quan la càrrega de computació per procés és molt baixa, l'overhead de sincronització domina i redueix l'eficiència global, fet coherent amb els resultats de strong scaling.

El dataset utilitzat (1934 patrons) és relativament petit. Amb 64 processos, cada procés només processa 30 patrons. Per datasets massius (>50K patrons), MPI podria escalar a centenars de processos mantenint una eficiència adequada, ja que la proporció computació/-comunicació augmentaria.

4.5 Treball Futur

Es poden explorar diverses optimitzacions per millorar l'escalabilitat: processament per parts més gran per augmentar la càrrega de computació, utilitzar versions d'*Allreduce* òptimes, aplicar comunicació potencialment asíncrona, ajustar la taxa d'aprenentatge segons el nombre de processos, i explotar GPU o entorns híbrids CPU/GPU per reduir el temps total i augmentar l'eficiència.

A més, l'estudi es podria ampliar explorant un rang més gran de valors per als paràmetres de la xarxa, com el nombre de neurones i *epochs*. Això permetria analitzar amb més detall

com escala realment el problema, especialment en experiments de *weak scaling*, on es manté constant la càrrega de treball per procés mentre s'incrementa el nombre total de processos. Això ajudaria a identificar límits d'escalabilitat i colls d'ampolla específics de la comunicació i sincronització en diferents configuracions, i a validar si les tendències observades amb el *dataset* actual es mantenen amb sistemes més grans o més intensius en càlcul.

5 Annex

5.1 Taules Completes de Resultats

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	3.1012	871	1.0000	1.0000
2	3.2115	876	0.9657	0.4828
4	2.0203	874	1.5350	0.3838
6	1.5539	856	1.9958	0.3326
8	1.2779	864	2.4268	0.3033
10	0.9665	852	3.2087	0.3209
12	0.8762	841	3.5394	0.2949
16	0.6564	846	4.7246	0.2953
20	0.6129	832	5.0599	0.2530
24	0.4785	778	6.4811	0.2700
32	0.3767	795	8.2325	0.2573
36	0.5391	782	5.7526	0.1598
40	0.3183	775	9.7430	0.2436
48	0.2832	691	10.9506	0.2281
64	0.2338	641	13.2643	0.2073

Taula 4: Partició `new-nodo.q`, 135 neurones, 10 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	31.4343	912	1.0000	1.0000
2	33.5528	917	0.9369	0.4684
4	18.3128	911	1.7165	0.4291
6	12.0636	911	2.6057	0.4343
8	10.6216	908	2.9595	0.3699
10	9.9517	907	3.1587	0.3159
12	8.6528	907	3.6328	0.3027
16	5.4300	902	5.7890	0.3618
20	7.8262	898	4.0165	0.2008
24	4.7098	899	6.6742	0.2781
32	6.7493	894	4.6574	0.1455
36	6.6003	895	4.7626	0.1323
40	3.0653	892	10.2549	0.2564
48	2.6320	886	11.9431	0.2488
64	2.1767	885	14.4413	0.2256

Taula 5: Partició `new-nodo.q`, 135 neurones, 100 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	5.8993	900	1.0000	1.0000
2	6.1559	895	0.9583	0.4792
4	3.5854	887	1.6454	0.4113
6	2.5880	878	2.2795	0.3799
8	2.2217	869	2.6553	0.3319
10	2.1068	865	2.8001	0.2800
12	3.5251	871	1.6735	0.1395
16	2.7256	868	2.1644	0.1353
20	0.8514	850	6.9289	0.3464
24	0.8025	849	7.3512	0.3063
32	1.4475	846	4.0755	0.1274
36	1.3149	841	4.4865	0.1246
40	1.1825	827	4.9888	0.1247
48	0.9651	786	6.1126	0.1273
64	0.8827	745	6.6832	0.1044

Taula 6: Partició `new-nodo.q`, 250 neurones, 10 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	58.2278	916	1.0000	1.0000
2	70.0410	917	0.8313	0.4157
4	32.9367	916	1.7679	0.4420
6	23.2260	920	2.5070	0.4178
8	17.3091	918	3.3640	0.4205
10	14.1683	922	4.1097	0.4110
12	12.0905	921	4.8160	0.4013
16	12.0206	919	4.8440	0.3028
20	12.3112	917	4.7297	0.2365
24	8.5598	912	6.8025	0.2834
32	14.2773	909	4.0783	0.1274
36	12.9740	907	4.4880	0.1247
40	11.4300	908	5.0943	0.1274
48	9.1771	907	6.3449	0.1322
64	7.8317	895	7.4349	0.1162

Taula 7: Partició `new-nodo.q`, 250 neurones, 100 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	10.3570	871	1.0000	1.0000
2	5.5925	876	1.8519	0.9260
4	2.8058	874	3.6913	0.9228
6	1.9933	856	5.1959	0.8660
8	1.5935	864	6.4995	0.8124
10	1.1925	852	8.6851	0.8685
12	1.0957	841	9.4524	0.7877
16	0.9901	846	10.4606	0.6538
20	0.7860	832	13.1768	0.6588
24	0.7205	778	14.3747	0.5989
32	1.4819	795	6.9890	0.2184
36	0.7538	782	13.7397	0.3817
40	0.6546	775	15.8219	0.3955
48	1.8645	691	5.5548	0.1157
64	2.7677	641	3.7421	0.0585

Taula 8: Partició `nodo.q`, 135 neurones, 10 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	107.9327	912	1.0000	1.0000
2	51.5186	917	2.0950	1.0475
4	26.3944	911	4.0892	1.0223
6	18.3426	911	5.8843	0.9807
8	16.1244	908	6.6937	0.8367
10	11.4037	907	9.4647	0.9465
12	9.4992	907	11.3623	0.9469
16	8.7962	902	12.2704	0.7669
20	8.5682	898	12.5969	0.6298
24	7.3571	899	14.6705	0.6113
32	9.1248	894	11.8285	0.3696
36	10.1401	895	10.6441	0.2957
40	7.3384	892	14.7079	0.3677
48	19.5853	886	5.5109	0.1148
64	27.1922	885	3.9693	0.0620

Taula 9: Partició nodo.q, 135 neurones, 100 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	19.1346	900	1.0000	1.0000
2	9.8299	895	1.9466	0.9733
4	6.2093	887	3.0816	0.7704
6	7.8608	878	2.4342	0.4057
8	5.6149	869	3.4078	0.4260
10	5.1335	865	3.7274	0.3727
12	4.5391	871	4.2155	0.3513
16	3.5269	868	5.4253	0.3391
20	2.9411	850	6.5059	0.3253
24	2.6040	849	7.3482	0.3062
32	2.8000	846	6.8338	0.2136
36	2.3947	841	7.9904	0.2220
40	1.9913	827	9.6091	0.2402
48	3.8383	786	4.9852	0.1039
64	5.4017	745	3.5423	0.0553

Taula 10: Partició nodo.q, 250 neurones, 10 èpoques

Nº Processos	Temps	Encerts	Speedup	Eficiència
1	188.7682	916	1.0000	1.0000
2	97.3295	917	1.9395	0.9697
4	55.9714	916	3.3726	0.8431
6	77.5987	920	2.4326	0.4054
8	59.0819	918	3.1950	0.3994
10	30.9059	922	6.1078	0.6108
12	42.4651	921	4.4453	0.3704
16	35.3729	919	5.3365	0.3335
20	26.0143	917	7.2563	0.3628
24	26.2810	912	7.1827	0.2993
32	29.8533	909	6.3232	0.1976
36	27.6855	907	6.8183	0.1894
40	23.2596	908	8.1157	0.2029
48	22.4182	907	8.4203	0.1754
64	43.5746	895	4.3321	0.0677

Taula 11: Partició nodo.q, 250 neurones, 100 èpoques

5.2 Codi

```
1 #include "main.h"
2 #include <fcntl.h>
3 #include <limits.h>
4 #include <math.h>
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <sys/time.h>
8 #include <mpi.h>
9
10 void freeInput(int np, char** input) {
11     for (int i = 0; i < np; i++)
12         free(input[i]);
13     free(input);
14 }
15
16 void printRecognized(int p, layer Output) {
17     int imax = 0;
18
19     for (int i = 1; i < num_out_layer; i++)
20         if (Output.actv[i] > Output.actv[imax])
21             imax = i;
22
23     if (imax == Validation[p])
24         total++;
25
26     if (debug == 1) {
27         printf("El patró %d sembla un %c\t i és un %d", p, '0' + imax,
28             Validation[p]);
29         for (int k = 0; k < num_out_layer; k++)
30             printf("\t%f\t", Output.actv[k]);
31         printf("\n");
32     }
33 }
34
35 void train_neural_net() {
36     double t0 = MPI_Wtime();
37
38     int extra = num_training_patterns % num_procs;
39     int from = rank * (num_training_patterns / num_procs) + ((rank < extra) ? rank :
40         extra);
41     int to = from + (num_training_patterns / num_procs) + ((rank < extra) ? 1 : 0);
42
43     if (debug)
44         printf("TRAIN Rank %d -> from %d to %d\n", rank, from, to);
45
46     if ((input = loadPatternSet(num_training_patterns, dataset_training_path, 1)) == NULL) {
47         printf("Loading Patterns: Error!!\n");
48     }
```

```
47         exit(-1);
48     }
49
50     int *ranpat = malloc(num_training_patterns * sizeof(int));
51
52     for (int it = 0; it < num_epochs; it++) {
53         if (rank == 0) {
54             for (int p = 0; p < num_training_patterns; p++)
55                 ranpat[p] = p;
56
57             for (int p = 0; p < num_training_patterns; p++) {
58                 int x = rand();
59                 int np = (x * x) % num_training_patterns;
60                 int tmp = ranpat[p];
61                 ranpat[p] = ranpat[np];
62                 ranpat[np] = tmp;
63             }
64         }
65
66         MPI_Bcast(ranpat, num_training_patterns, MPI_INT, 0, MPI_COMM_WORLD);
67
68         for (int i = from; i < to; i++) {
69             int p = ranpat[i];
70             feed_input(p);
71             forward_prop();
72             back_prop(p);
73             update_weights();
74         }
75
76         for (int l = 0; l < num_layers - 1; l++) {
77             long size = num_neurons[l] * num_neurons[l+1];
78
79             MPI_Allreduce(MPI_IN_PLACE, lay[l].out_weights, size, MPI_FLOAT, MPI_SUM,
80                           MPI_COMM_WORLD);
81             for (long i = 0; i < size; i++)
82                 lay[l].out_weights[i] /= num_procs;
83
84             MPI_Allreduce(MPI_IN_PLACE, lay[l].bias, num_neurons[l], MPI_FLOAT, MPI_SUM,
85                           MPI_COMM_WORLD);
86             for (int i = 0; i < num_neurons[l]; i++)
87                 lay[l].bias[i] /= num_procs;
88         }
89
90         free(ranpat);
91         freeInput(num_training_patterns, input);
92
93         double local = MPI_Wtime() - t0;
94         MPI_Reduce(&local, &train_t, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
```

```
94 }
95
96 void test_nn() {
97     double t0 = MPI_Wtime();
98
99     char** rSet;
100
101     if ((rSet = loadPatternSet(num_test_patterns, dataset_test_path, 0)) ==
102         NULL) {
103         printf("Error!!\n");
104         exit(-1);
105     }
106
107     int extra = num_test_patterns % num_procs;
108     int from = rank * (num_test_patterns / num_procs) + ((rank < extra) ? rank : extra);
109     int to = from + (num_test_patterns / num_procs) + ((rank < extra) ? 1 : 0);
110
111     for (int i = from; i < to; i++) {
112         for (int j = 0; j < num_neurons[0]; j++)
113             lay[0].actv[j] = rSet[i][j];
114
115         forward_prop();
116         printRecognized(i, lay[num_layers - 1]);
117     }
118
119     int global_total = 0;
120     MPI_Allreduce(&total, &global_total, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
121     total = global_total;
122
123     double local = MPI_Wtime() - t0;
124     MPI_Reduce(&local, &test_t, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
125
126     freeInput(num_test_patterns, rSet);
127 }
128
129 int main(int argc, char** argv) {
130     MPI_Init(&argc, &argv);
131
132     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
133     MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
134
135     if (debug == 1)
136         printf("argc = %d \n", argc);
137
138     if (argc == 2) {
139         readConfiguration(argv[1]);
140     } else {
141         readConfiguration("configuration/configfile.txt");
142     }
```

```
143
144     if (argc == 3) {
145         num_epochs = atoi(argv[1]);
146         num_neurons[1] = atoi(argv[2]);
147     }
148
149     if (debug == 1)
150         printf("FINISH CONFIG \n");
151
152     if (init() != SUCCESS_INIT) {
153         printf("Error in Initialization...\n");
154         exit(0);
155     }
156
157     if (debug == 1)
158         printf("COST MALLOC \n");
159
160     cost = (float*)malloc(num_neurons[num_layers - 1] * sizeof(float));
161
162     train_neural_net();
163     test_nn();
164
165     if (rank == 0) {
166         printf("RIGHT %d\n", total);
167         printf("ACCUR %6.4f\n", (float) total / num_test_patterns);
168         printf("TRAIN %6.4f\n", train_t);
169         printf("TEST %6.4f\n", test_t);
170         printf("TOTAL %6.4f\n", train_t + test_t);
171     }
172
173     if (dinit() != SUCCESS_DINIT)
174         printf("Error in Dinitialization...\n");
175
176     free(cost);
177
178     MPI_Finalize();
179
180     return 0;
181 }
```

5.3 Scripts

- run.py

```
1 import argparse
2 import subprocess
3 import re
```

```
4 from time import sleep
5 import os
6
7 # Constants
8 SCHEDULER_FILE = "mpi.sub"
9
10 # Arguemnts
11 parser = argparse.ArgumentParser()
12 parser.add_argument("--name", type=str, default="slurm")
13 parser.add_argument("--nprocs", type=int, default=8)
14 parser.add_argument("--partition", type=str, default="nodo.q")
15 parser.add_argument("--num_epochs", type=int, default=10)
16 parser.add_argument("--num_neurons", type=int, default=250)
17 parser.add_argument("--repeat", type=int, default=1)
18 parser.add_argument("--open", dest="open", action="store_true")
19 parser.add_argument("--no-open", dest="open", action="store_false")
20 parser.set_defaults(open=True)
21 args = parser.parse_args()
22
23 # Change scheduler file
24 with open(SCHEDULER_FILE, "r") as f:
25     contents = f.read()
26     contents = re.sub(r"#SBATCH\s+-o\s+\S+", f"#SBATCH -o OUT/{args.name}-%j.out", contents)
27     contents = re.sub(r"#SBATCH\s+-e\s+\S+", f"#SBATCH -e OUT/{args.name}-%j.err", contents)
28     contents = re.sub(r"#SBATCH\s+--ntasks=\d+", f"#SBATCH --ntasks={args.nprocs}", contents)
29     contents = re.sub(r"#SBATCH\s+--partition=\S+", f"#SBATCH --partition={args.partition}",
30         contents)
31 with open(SCHEDULER_FILE, "w") as f:
32     f.write(contents)
33
34 # Submit job
35 res = subprocess.run(
36     ["sbatch", SCHEDULER_FILE, str(args.num_epochs), str(args.num_neurons),
37         str(args.repeat)],
38     stdout=subprocess.PIPE,
39     stderr=subprocess.PIPE,
40     universal_newlines=True
41 )
42 print(res.stdout.strip())
43
44 # Open output file
45 if args.open:
46     job_id = re.search(r"Submitted batch job (\d+)", res.stdout.strip()).group(1)
47
48     while True:
49         res = subprocess.run(
50             ["squeue", "-h", "-j", str(job_id), "-o", "%i", "-u", os.environ.get("USER",
51                 "capmc-1")],
52             stdout=subprocess.PIPE,
```

```
50         stderr=subprocess.PIPE,
51         universal_newlines=True
52     )
53     if str(job_id) not in res.stdout:
54         break
55     sleep(0.1)
56
57     subprocess.run(["code", f"OUT/{args.name}-{job_id}.out"])
```

- `mpi.sub`

```
1 #!/bin/bash
2 #SBATCH -o OUT/slurm-%j.out
3 #SBATCH -e OUT/slurm-%j.err
4 #SBATCH --ntasks=12
5 #SBATCH --partition=nodo.q
6
7 module purge
8 module load openmpi/4.1.1
9
10 # ARGUMENTS
11 NUM_EPOCHS=${1:-10}
12 NUM_NEURONS=${2:-250}
13 REPEAT=${3:-1}
14
15 # METADATA
16 echo "---"
17 echo "date: $(date '+%Y-%m-%d %H:%M:%S')"
18 echo "job_id: $SLURM_JOB_ID"
19 echo "repeat: $REPEAT"
20 echo "server: Wilma"
21 echo "partition: $SLURM_JOB_PARTITION"
22 echo "nprocs: $SLURM_NTASKS"
23 echo "num_epochs: $NUM_EPOCHS"
24 echo "num_neurons: $NUM_NEURONS"
25 echo "---"
26
27 # NETEJA
28 rm -f exec *.o common/*.o configuration/*.o layer/*.o randomizer/*.o initialize/*.o
29   training/*.o
30 rm -rf ./*.so ./*.a
31
32 # COMPILACIÓ
33 mpicc -O3 \
34     main.c \
35     common/common.c \
36     configuration/config.c \
37     layer/layer.c \
```

```
37 randomizer/randomizer.c \  
38 initialize/initialize.c \  
39 training/training.c \  
40 -o exec \  
41 -lm  
42  
43 # EXECUCIÓ  
44 for ((i=1; i<=$REPEAT; i++)); do  
45     echo ">>>>>>>>>> $i"  
46     mpirun -np $SLURM_NTASKS ./exec $NUM_EPOCHS $NUM_NEURONS  
47 done  
48 echo ">>>>>>>>>> -"
```

- makefile

```
1 OUTPUT_DIR=/home/alumnos/capmc/capmc-1/Escritorio/FFNN-SourceCode/OUT  
2 OUTFILE_PREFIX=slurm-  
3 ERRFILE_PREFIX=slurm-  
4  
5 PYTHON=python3  
6 RUN_SCRIPT=run.py  
7  
8 PARTITIONS := nodo.q new-nodo.q  
9 NPROCS := 2 4 6 8 10 12 16 20 24 32 36 40 48 64  
10 NUM_EPOCHS := 10 100  
11 NUM_NEURONS := 135 250  
12 REPEAT=5  
13  
14 all:  
15     @jid=$(sbatch mpi.sub | awk '{print $4}'); \  
16     echo "Submitted job $jid"; \  
17     while squeue -h -j $jid | grep -q "$jid"; do sleep 0.2; done; \  
18     outf="$(OUTPUT_DIR)/$(OUTFILE_PREFIX)$jid.out"; \  
19     errf="$(OUTPUT_DIR)/$(ERRFILE_PREFIX)$jid.err"; \  
20     code $$outf  
21  
22 test1:  
23     @echo "Starting batch runs..."  
24     @for p in $(PARTITIONS); do \  
25         for n in $(NPROCS); do \  
26             for h in $(NUM_EPOCHS); do \  
27                 for nn in $(NUM_NEURONS); do \  
28                     echo "Running: partition=$p nprocs=$n num_epochs=$h num_neurons=$nn  
29                         repeat=$(REPEAT)"; \  
30                     $(PYTHON) $(RUN_SCRIPT) --name test1 --partition $p --nprocs $n --num_epochs $h  
31                     --num_neurons $nn --repeat $(REPEAT) --no-open; \  
32                 done; \  
29             done; \  
30         done; \  
31     done;
```

```
32     done; \
33     done
```